

Unit 3 – Synchronization (Simplified Detailed Notes)

These notes explain all topics and subtopics from Unit 3 in a much simpler and more detailed way. Complex concepts are explained using easy language, real-life examples, and step-by-step flow so they are easier to understand and remember for exams.

Introduction to Synchronization

Synchronization means coordinating multiple processes or computers so they work together correctly.

In distributed systems, many processes may:

- Share resources
- Exchange messages
- Perform operations together

Synchronization ensures:

- Processes do not interfere with each other
- Shared resources are used safely
- Events occur in the correct order

Example:

Suppose 2 students try to print different assignments using the same printer at the same time.

Without synchronization:

- Pages may mix
- Output may become corrupted

Synchronization ensures:

- One student uses printer first
- Other waits until printer becomes free

Another Example:

Two bank servers update the same account balance.

Synchronization ensures updates happen correctly and consistently.

Why Synchronization is Difficult in Distributed Systems

In centralized systems:

- There is one system clock
- Memory is shared
- Processes can easily coordinate

In distributed systems:

- Different machines have different clocks
- No shared memory exists
- Communication delays occur
- Machines may fail independently

This makes synchronization much harder.

Example:

A message sent from Mumbai to Delhi may arrive later than expected because of network delays.

Importance of Clock Synchronization

Clock synchronization is extremely important in distributed computing.

Why?

Many systems depend on accurate timestamps.

Examples:

- Banking systems
- File systems
- Security auditing
- Distributed databases
- Stock market systems

Example with UNIX make Program:

Suppose:

input.c = modified at 2151

input.o = modified at 2150

Since input.c is newer:

The file must be recompiled.

Problem in Distributed Systems:

If different computers have unsynchronized clocks:

- A newer file may appear older
- Wrong decisions are made

Result:

- Incorrect executable program
- Possible crashes

Therefore:

Machines in distributed systems need synchronized clocks.

Physical Clocks

Every computer contains a hardware clock.

Main Component:

Quartz crystal oscillator.

How It Works:

- Crystal vibrates at fixed frequency
- Each vibration decreases counter
- When counter reaches zero:
 - Clock interrupt generated

This interrupt is called a clock tick.

Typical systems generate:

60–100 ticks per second.

Clock Drift:

Computer clocks are not perfect.

Their speed slightly changes over time.

Causes:

- Temperature changes
- Crystal imperfections
- Hardware differences

Clock Skew:

Difference between clocks of two machines.

Example:

Machine A:

10:00:00

Machine B:

09:59:58

Difference = clock skew.

Why It Matters:

Incorrect timestamps may:

- Corrupt distributed databases
- Cause wrong ordering
- Affect real-time systems

UTC and Time Synchronization

To maintain accurate time:

Computers synchronize with UTC (Universal Coordinated Time).

UTC is provided through:

- Atomic clocks
- Radio signals
- Satellites

Accuracy:

- Radio synchronization → milliseconds
- Satellite synchronization → nanoseconds

Example:

GPS systems use extremely accurate time synchronization.

Clock Synchronization Algorithms

Distributed systems use special algorithms to synchronize clocks.

Two important algorithms:

1. Network Time Protocol (NTP)
2. Berkeley Algorithm

Network Time Protocol (NTP)

NTP is one of the most widely used clock synchronization protocols.

Purpose:

Synchronize computers with accurate time servers.

Problem:

Network delays affect accuracy.

Working:

1. Client sends request at time T1.
2. Server receives request at T2.
3. Server replies at T3.
4. Client receives reply at T4.

Using these timestamps:

Client estimates:

- Network delay
- Clock difference

Important Idea:

Clocks should not suddenly jump backward.

Instead:

They adjust gradually.

Example:

Instead of adding 10 ms every tick:

System may temporarily add:

- 9 ms (slow down)

OR

- 11 ms (speed up)

NTP Strata:

Stratum-0:

Atomic clocks / GPS clocks

Stratum-1:

Directly connected to reference clocks

Stratum-2:

Synchronize from stratum-1 servers

Higher levels continue similarly.

Advantages:

- Accurate
- Scalable
- Internet-wide usage

Example:

Most modern operating systems use NTP.

Berkeley Algorithm

Berkeley Algorithm works differently from NTP.

In NTP:

Clients ask time server for time.

In Berkeley:

Time daemon actively asks all machines for their times.

Working:

1. Time daemon polls all machines.
2. Machines send local clock values.
3. Daemon calculates average time.
4. All systems adjust clocks gradually.

Important:

This algorithm does not require external UTC clock.

Main Goal:

All machines should agree on same time.

Example:

In a private company network:

Exact world time may not matter.

Only internal synchronization matters.

Properties Needed in Distributed Systems

Distributed systems must satisfy certain important properties.

1. Information is distributed across machines.
2. Processes use local information only.
3. No single point of failure should exist.
4. No common global clock exists.

These properties make synchronization challenging.

Logical Clocks

Sometimes exact physical time is unnecessary.

What matters more:

Correct ordering of events.

Example:

For make program:

Only need to know:

Which file is newer.

Not exact creation time.

Logical clocks solve this problem.

Main Idea:

Track order of events instead of real-world time.

Lamport's Logical Clocks

Lamport introduced the concept of logical clocks.

Main Goal:

Maintain correct event ordering.

Important Concept:

Happens-before relation.

Event a happens before event b if:

1. Both occur in same process and a occurs first.
2. a is message sending and b is message receiving.

Concurrent Events:

If two events are unrelated:

Their order cannot be determined.

Lamport Timestamp Rule:

If:

$a \rightarrow b$

Then:

$C(a) < C(b)$

Logical clocks always move forward.

Lamport Clock Algorithm

Each process maintains a logical counter.

Algorithm:

1. Before every event:

Increment local clock.

2. While sending message:

Attach timestamp.

3. On receiving message:

Receiver updates clock using:

$\max(\text{local clock}, \text{received timestamp})$

Then increments its clock.

Purpose:

Maintain consistent event ordering.

Example:

Suppose:

P1 sends message with timestamp 20.

P2 local clock = 15.

P2 updates:

$\max(15, 20) = 20$

Then increments to 21.

Benefits:

- Simple
- Distributed
- Maintains ordering

Totally Ordered Multicasting

Distributed databases often maintain multiple replicas.

Problem:

Updates may arrive in different orders.

Example:

Account balance = \$1000

Operation 1:

Deposit \$100

Operation 2:
Add 1% interest

If order differs:
Results become inconsistent.

Case 1:
Deposit first:
 $1100 \rightarrow 1\% \text{ interest} = 1111$

Case 2:
Interest first:
 $1010 \rightarrow \text{deposit} = 1110$

Different replicas produce different balances.

Solution:
Totally Ordered Multicast.

Goal:
All replicas execute operations in same order.

Implementation of Totally Ordered Multicast

Working:

1. Every multicast message gets Lamport timestamp.
2. Message also sent to sender itself.
3. Receiver stores messages in timestamp order queue.
4. Receiver sends acknowledgment to everyone.

Message delivered only when:
- It is at front of queue
- Acknowledged by all processes

Result:
All systems process messages in identical order.

Used In:
- Replicated databases
- Distributed banking systems
- State machine replication

Vector Clocks

Problem with Lamport clocks:

They maintain ordering but cannot fully capture causality.

Example:

Two independent events may still appear ordered.

Solution:

Vector clocks.

Each process maintains vector:

[events in P1, events in P2, events in P3...]

Meaning:

$VC_i[j]$ = how many events process P_i knows occurred at P_j .

Vector Clock Algorithm

Algorithm:

1. Before event:

Increment own position in vector.

2. While sending message:

Attach entire vector.

3. On receiving:

Take maximum of local vector and received vector.

Then increment own value.

Advantages:

- Tracks causality accurately
- Detects concurrent events
- Better than Lamport clocks

Example:

If:

$VC(a) < VC(b)$

Then:

a causally happened before b.

Mutual Exclusion

Mutual exclusion ensures only one process uses shared resource at a time.

Shared resources:

- Printer
- File
- Database record

Critical Section (CS):

Code section accessing shared resource.

Goal:

Prevent concurrent access.

In distributed systems:

Shared memory does not exist.

Therefore:

Message passing is used.

Requirements of Mutual Exclusion

A good mutual exclusion algorithm must satisfy:

1. Safety

Only one process enters CS at a time.

2. Liveness

Every requesting process eventually enters CS.

No:

- Deadlock
- Starvation

3. Fairness

Every process gets fair chance.

Performance Metrics:

- Synchronization delay
- Throughput
- Message complexity
- Response time

Centralized Mutual Exclusion Algorithm

Simplest distributed mutual exclusion method.

Idea:

One coordinator controls access.

Working:

1. Process sends request.
2. Coordinator grants permission if resource free.
3. If busy:
Request queued.
4. After use:
Process sends release message.

Advantages:

- Simple
- Fair
- Easy to implement

Requires:

3 messages:

Request → Grant → Release

Drawbacks:

- Coordinator failure stops system
- Bottleneck in large systems

Lamport's Distributed Mutual Exclusion Algorithm

Fully distributed approach.

Main Idea:

Use Lamport timestamps for priority.

Smaller timestamp = higher priority.

Working:

1. Process broadcasts REQUEST.
2. All processes send REPLY.
3. Requests stored in queue by timestamp.
4. Process enters CS only if:
 - Received replies from everyone
 - Its request at queue front

After leaving CS:

- Broadcast RELEASE message.

Advantages:

- Fair ordering
- No coordinator

Disadvantages:

- Large message overhead
- Requires reliable communication

Ricart–Agrawala Algorithm

Improved distributed mutual exclusion algorithm.

Improvement:

Reduces message count.

Working:

1. Process sends REQUEST to all.
2. Receiver checks:
 - Not interested → send OK
 - Using resource → delay reply
 - Also wants resource → compare timestamps

Lower timestamp wins.

After exiting CS:

Delayed replies are sent.

Advantages:

- Fully distributed
- No starvation
- Better than Lamport algorithm

Disadvantages:

- Process crash may block system
- High communication overhead

Token Ring Algorithm

Processes arranged logically in ring.

Main Idea:

Only process holding token may enter CS.

Working:

1. Token circulates around ring.
2. Process receives token.
3. If resource needed:
Enter CS.
4. After completion:
Pass token.

Advantages:

- Mutual exclusion guaranteed
- No starvation
- Very fair

Problems:

- Lost token difficult to detect
- Process crash breaks ring

Comparison of Mutual Exclusion Algorithms

1. Centralized Algorithm

- Simple
- Low message cost
- Single point of failure

2. Lamport Algorithm

- Fully distributed
- High message complexity

3. Ricart–Agrawala

- Fewer messages than Lamport
- Still sensitive to crashes

4. Token Ring

- Very fair
- Efficient when token available
- Token loss problematic

Election Algorithms

Distributed systems often require leader/coordinator.

Election algorithms choose leader.

Assumptions:

- Each process has unique ID.
- Highest ID usually becomes leader.
- Processes know IDs of others.

Bully Algorithm

Main Idea:

Highest numbered active process becomes coordinator.

Working:

1. Process detects coordinator failure.
2. Sends ELECTION messages to higher IDs.
3. If nobody replies:
It becomes coordinator.
4. If higher process replies:
Higher process takes over election.

Eventually:

Highest active process wins.

Why called Bully?

Because strongest (highest ID) process always wins.

Advantages:

- Simple
- Fast when high-ID processes alive

Disadvantages:

- Many messages
- High overhead

Ring Election Algorithm

Processes arranged in logical ring.

Working:

1. Process notices coordinator failure.
2. Sends ELECTION message around ring.
3. Each process adds its ID.
4. Message eventually returns.

Highest ID becomes coordinator.

Then:

COORDINATOR message circulates informing everyone.

Advantages:

- Distributed
- No central authority

Disadvantages:

- Slower than Bully algorithm
- Ring maintenance required

Exam Tip: Focus on understanding the flow of algorithms step-by-step instead of memorizing them mechanically. Topics like Lamport clocks, vector clocks, mutual exclusion, and election algorithms become much easier when visualized using real-life coordination examples.